

Obs from the EXAM! (2026) 1) Stair * s1 = new WoodenStair{...}, not Stair s1 = new WoodenStair{...} 2) always put ... = nullptr at the end (as the last parameter!) in case you skip it, it knows exactly which one got skipped!

class Parent {
private: int x; int y;
public:
Parent(int x, int y):
x{x}, y{y} {}
};

class Child : public Parent {
public:
Child(int x, int y):
Parent{x, y} {}
};

- 3) even if it was nullptr, you still had to create the metalhandrail
4) constructor with nr. of steps even if it inherits (but no private fields)
5) you could call the class Handrail{handrail...} in the init. list of the constructor of the child class, but in that case you have to provide an appropriate constructor (not a default one?), pay attention if you do handrail{handrail} or Handrail{handrail} whatever...
6) you don't always need to declare a variable in private in order for you to use a variable in the constructor!

3. Beverage

a) class Beverage {

private:

std::string description;

public:

Beverage(const ...): description{description} {}

virtual double price() const = 0;

virtual void print() const {

cout << description << " " << price << "\n";

}

virtual ~Beverage() = default;

};

b) class Coffee : public Beverage {

public:

Coffee(): Beverage{"Coffee"} {}

};

class Tea : public Beverage {

public:

Tea(): Beverage{"Tea"} {}

};

by calling the base class constructor with the descriptions "Coffee", "Tea".

c) class BeverageWithMilk : public Beverage {

private:

Beverage* beverage;

int milkCount;

public:

BeverageWithMilk(Beverage* beverage, int milkCount): Beverage{"with milk"},

beverage{beverage},

milkCount{milkCount} {}

double price() const override {

beverage -> price() + milkCount * 0.5;

}

void print() override {

beverage -> print();

std::cout << "portions: " << milkCount << "\n";

}

~BeverageWithMilk() override {

delete beverage;

}

};

only for parent-child situations!

d) class BeverageMachine {

public:

void prepare (const std::string& beverageType, int milkCount) {

Beverage* beverage;

if (beverageType == "Tea")
beverage = new Tea {};

else
beverage = new Coffee {};

if (milkCount > 0) {

beverage = new BeverageWithMilk (beverage, milkCount);

beverage->print();

std::cout << "Price: " << beverage->price() << "\n";

delete beverage;

};

3. Telephone

a) class Channel {

public:

virtual void send (const std::string& message) = 0;

virtual ~Channel () = default;

};

b) class Telephone: public Channel {

private:

std::string number;

public:

Telephone (const ...): number { number } {}

void send (...) override {

int k = rand () % 10;

if (k == 9) {

throw std::runtime_error ("Line busy");

std::cout << "dialing" << number << "\n";

};

c) class Fax: public Telephone {

~~public: private:~~

~~std::string number;~~

public:

Fax (const ...): Telephone { number } {}

void send (...) override {

Telephone::send (message);

std::cout << "sending ~~msg~~" << message << "\n";

};

I did not put the nr. as a private field, so I call the constructor of the base class for it.


```

class SHS: public SHS Telephone {
public:

```

```

    SHS(---): Telephone { number } {}

```

```

    void send(...) override {

```

```

        Telephone::send(-);

```

```

        std::cout << "sending SHS ..." << message << "\n";
    }
};

```

```

d) class Failover: public Channel {

```

```

private:

```

```

    Channel* first;

```

```

    Channel* second;

```

```

public:

```

```

    Failover(Channel* first, Channel* second): first{first},
                                                second{second} {}

```

```

    void send(...) override {

```

```

        try {
            first->send(-);

```

```

        }
        catch (----) {

```

```

            second->send(-);

```

```

        }
    }

```

```

    ~Failover() override {

```

```

        delete first;

```

```

        delete second;
    }
};

```

```

e) class Contact {

```

```

private:

```

```

    Channel* c1;

```

```

    Channel* c2;

```

```

    Channel* c3;

```

```

public:

```

```

    Contact(Channel* c1, Channel* c2, Channel* c3): c1{c1}, c2{c2}, c3{c3} {}

```

```

    void sendAlarm(-----) {

```

```

        try {

```

```

            c1->send(message);

```

```

            return;

```

```

        }

```

```

        catch (...) {

```

```

            try {

```

```

                c2->send(message);

```

```

            }
            return;

```

```

        }
        catch (...) {

```

```

            try {

```

```

                c3->send(message);

```

```

            }
            return;

```

```

        }
        catch (...) {

```

```

            throw std::runtime_error("All channels failed");
        }
    }
};

```



```
f) Contact* createContact(const std::string& number){
```

```
    Channel* ch1 = new Telephone {number};
```

```
    Channel* ch2 = new Failover {new Fax {number}, new SMS {number}}
```

```
    Channel* ch3 = new Failover {new Telephone {number},  
                                new Failover {new Fax {number},  
                                                new SMS {number}}};
```

```
    return new Contact {ch1, ch2, ch3};
```

3. Sale

```
a) class SaleItem {
```

```
    private:
```

```
        int code;
```

```
        double price;
```

```
    public:
```

```
        SaleItem (.....): ..... {}
```

```
        int getCode()...
```

```
        double getPrice()...
```

```
};
```

```
b) class Sale {
```

```
    private:
```

```
        std::vector<SaleItem* > items;
```

```
    public:
```

```
        void
```

```
        const ..... getItems()...
```

```
};
```

```
b) class Discount {
```

```
    public:
```

```
        virtual double calc (const Sale& s) const = 0;
```

```
        virtual ~Discount () = default;
```

```
};
```

```
c) class ItemDiscount : public Discount {
```

```
    private:
```

```
        int code, percentage;
```

```
    public:
```

```
        ItemDiscount (.....) ..... {}
```

```
        double calc (....) ... override {
```

```
            double result = 0;
```

```
            for (const auto& item: s.getItems()) {
```

```
                if (item.getCode() == code)
```

```
                    result += ...
```

```
            return result;
```

```
};
```

```
};
```

```
};
```

```
};
```



```

d) class SumDiscount: public Discount {
private:
    std::vector<Discount*> discounts;
public:
    SumDiscount
    void add (Discount* d) {
        discounts.push_back(d);
    }
    double Calc(const Sale& s) ... override {
        ....
    }
    ~SumDiscount() override {
        for(auto d: discounts)
            delete d;
    }
};

```

```

e) int main() {
    Sale sale;
    sale.addItem ....
    ....
    SumDiscount discount;
    discount.add (new ItemDiscount {1, 10})
    discount.add (new ItemDiscount {2, 15})
    ....
}

```

f)

3. Action, Menu

```
a) class Action {
    public:
        virtual void execute() = 0;
        virtual ~Action() = default;
};

class CreateAction: public Action {
    public:
        void execute() override {
            std::cout << "create file";
        }
};

class ExitAction: public Action {
    public:
        void execute() override {
            std::cout << "exit app";
        }
};
```

```
b) class MenuItem {
    private:
        std::string text;
        Action* action;
    public:
        MenuItem(const std::string& text, Action* action) :
            text{text}, action{action} {}

        void print() {
            std::cout << text;
        }

        void clicked() {
            std::cout << text;
            if (action != nullptr)
                action->execute();
        }

        virtual ~MenuItem() {
            delete action;
        }
};
```

last field! (optional)

nullptr

class Menu: public MenuItem {

private:

std::vector<MenuItem*^{0.*}> ~~menus~~ items;

public:

Menu(const std::string& text, Action* action = nullptr):

MenuItem { text, action } {}

void add(MenuItem* item) {

items.push_back(item);

}

void print() {

std::cout << "text" << "\n";

for(auto item: items)

item->print();

}

~Menu() override {

for(auto item: items)

delete item;

}

}

d) class MenuBar {

private:

std::vector<Menu*> menus;

public:

void add(Menu* menu) {

menus.push_back(menu);

}

void print() {

for(auto menu: menus)

menu->print();

}

~MenuBar() {

for(auto menu: menus)

delete menu;

}

}

e) `int main() {`

`MenuBar menuBar;`

auto* {

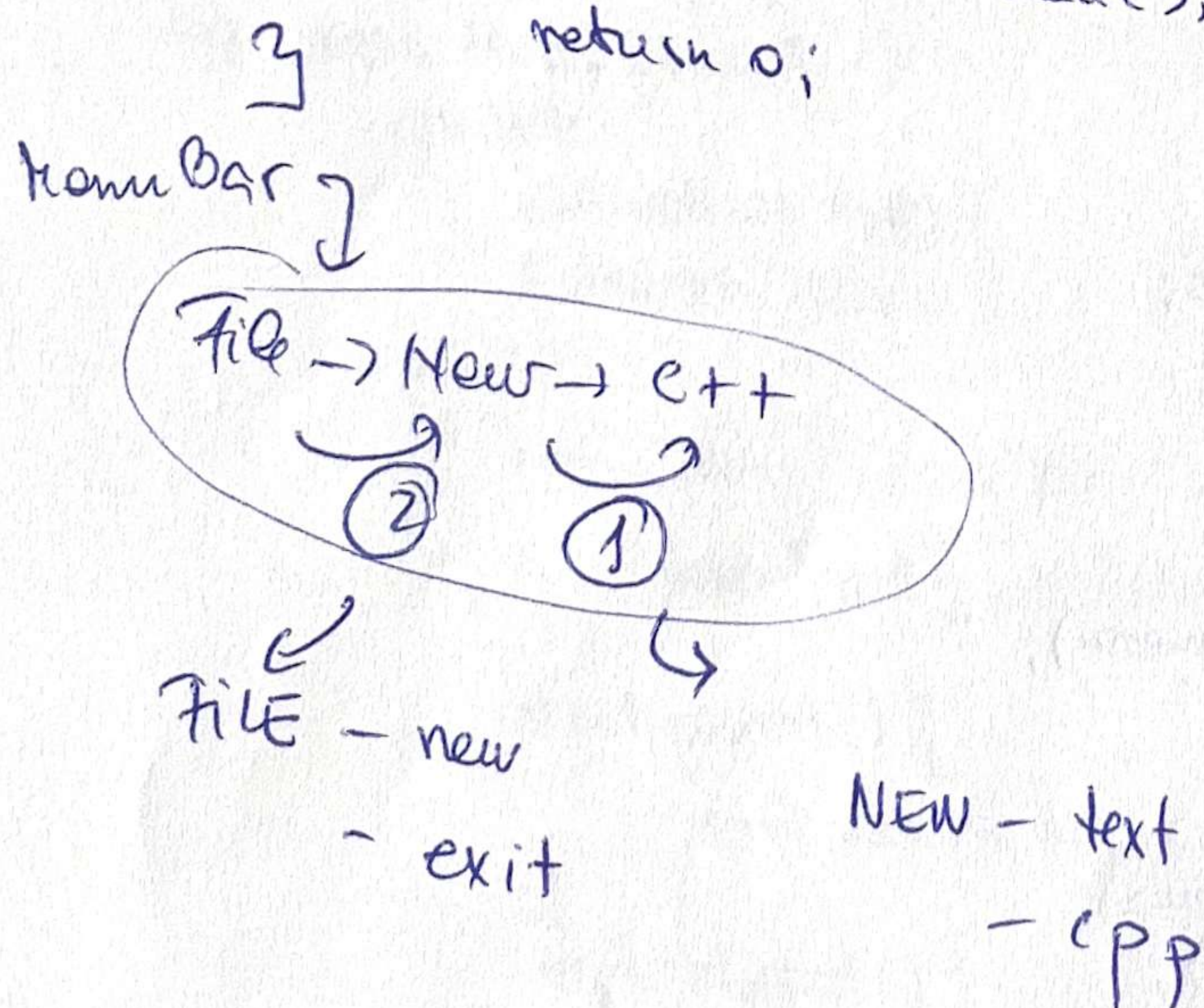
```

MenuItem* file = new Menu { "File" };
MenuItem* about = new Menu { "About" }; ✓
MenuItem* newMenu = new Menu { "New" }; ✓
MenuItem* exitMenu = new Menu { "Exit", new ExitAction{} }; ✓
MenuItem* text = new Menu { "Text", new CreateAction{} }; ✓
MenuItem* cpp = new Menu { "C++", new CreateAction{} }; ✓

newMenu → add(text);
newMenu → add(cpp); ) ①
file → add(newMenu);
file → add(exitMenu); ) ②

menuBar.add(file);
menuBar.add(about); ) ③
menuBar.print();
newMenu → get clicked();
exitMenu → clicked();
return 0;

```



3. Icecream

a) `class Icecream {`

```

virtual std::string getDescription() const = 0;
virtual double computePrice() = 0;
virtual ~Icecream() = default;

```



```

class SimpleIcecream : public Icecream {
private:
    std::string description;
    double price;
public:
    SimpleIcecream(const __, double __) : ____ {}
    std::string getDescription() override {
        std:: return "simple ic with" + description;
    }
    double computePrice() override {
        return price;
    }
};

```

```

c) class IcecreamWithTopping : public Icecream {
protected:
    Icecream* icecream;
public:
    IcecreamWithTopping(Icecream* icecream) : icecream{icecream} {}
    virtual std::string addTopping() const = 0;
    std::string getDescription() override {
        return icecream->getDescription() + icecream getTopping();
    }
    ~IcecreamWithTopping() override {
        delete icecream;
    }
};

```

```

d) class IcecreamWithChocolateTopping : public IcecreamWithTopping {
public:
    Ice__ (Icecream* icecream) : IcecreamWithTopping {icecream} {}

```

```

e) class Order {
private:
    std::vector< Icecream* > icecreams;
public:
    void add __
    void printOrder() {
        std::sort( icecreams.begin(), icecreams.end(), [](
            Icecream* a, Icecream* b) ____
    }

```



```

2) int main() {
    Order order;
    order.addIcecream(new Simple ~ {"vanilla", 5});
    ~~~~~ (new Ice ~ Caramel {new Ice ~ choc {
        new Simple ~ {"pistachio", 6} } } );
    ~~~~~ (new Slice Topping Caramel {new Simple ~ {
        "chocolate", 7} } );
    ~~~~~ (new SimpleIcecream {"hazelnuts", 8});
    order.printOrder();
}

```

3. Widgets (916)

```

a) class Event {
    public:
        virtual void trigger() = 0;
        virtual ~Event() = default;
};

class ClickEvent: public Event {
    public:
        void trigger() override {
            std::cout << "Clicked event triggered";
        }
};

class ToggleEvent: public Event {
    public:
        void trigger() override {
            std::cout << "Toggle event triggered";
        }
};

```

```

b) class Widget {
    private:
        std::string label;
        Event* event;
    public:
        Widget(const std::string& label, Event* event):
            label{label}, event{event} {}
        void render() {
            std::cout << label;
        }
        void interact() {
            std::cout << label;
            if (event != nullptr) event->trigger();
        }
};

```

✓ does not inherit => don't do Event{...}

= nullptr


```
virtual ~Widget() {
    delete event;
}
```

```
c) class Container: public Widget {
    private:
        std::vector<Widget*> widgets;
```

not private fields, but
from the parent class!

public:

```
Container(const std::string& label, Event* event = nullptr):
    Widget(label, event) {}
```

```
void add(Widget* w) {
    widgets.push_back(w);
}
```

```
void render() {
    std::cout << label;
    for (auto w: widgets)
        w->render();
}
```

```
~Container() override {
    for (auto w: widgets)
        delete w;
}
```

```
};
```

```
d) class GUI {
```

private:

```
std::vector<Container*> containers;
```

public:

```
void add(Container* c) {
    containers.push_back(c);
}
```

```
void render() {
    for (auto c: containers)
        c->render();
}
```

```
~GUI() {
    for (auto c: containers)
        delete c;
}
```

```
};
```


e) int main() {

GUI gui;

auto* MainPanel = new Container { "Main Panel" };

auto* HelpPanel = new Container { "Help Panel" };

auto* ExitButton = new Container { "Exit Button", new ToggleEvent {} };

auto* TextButton = new Container { "Text Button", new ClickedEvent {} };

auto* CppButton = new Container { "Cpp Button", new ClickedEvent {} };

auto* SettingsPanel = new Container { "Settings Panel" };

SettingsPanel → add(TextButton);
SettingsPanel → add(CppButton);

MainPanel → add(SettingsPanel);

MainPanel → add(ExitButton);

gui.add(MainPanel);

gui.add(HelpPanel);

CppButton → interact();

ExitButton → interact();

return 0;

}

MainP → SettingsP → Cpp Button

3. MathHur

a) class Expression {

public:

virtual double evaluate() = 0;

};

virtual ~Expression = default;

b) class Constant: public Expression {

private:

double value;

public:

Constant(double value): value {value} {}

double evaluate() override {

return value;

};

c) class UnaryExpression : public Expression {
protected:

Expression* right;

public:

UnaryExpression(Expression* right): right {right} {}

double evaluate() override {
return right->evaluate();
}

~UnaryExpression() override {
delete right;
}

};

class Negate: public UnaryExpression {

public:

Negate(Expression* expr): UnaryExpression {expr} {}

double evaluate() override {
return -expr->evaluate();
}

};

d) class BinaryExpression: public UnaryExpression {
protected:

Expression* ~~expr~~ left;

public:

BinaryExpression(Expression* left, Expression* right):

UnaryExpression {right}, left {left} {}

virtual ~BinaryExpression() {
delete left;
}

};

e) class Adder: public BinaryExpression {

public:

Adder(Expression* left, Expression* right): BinaryExpression {left,
right} {}

double evaluate() ~~const~~ override {
return left->evaluate() + right->evaluate();
}

};

from
grandparent


```
class Subtractor : public BinaryExpression {
```

```
public:
```

```
    Subtractor (Expression* left, Expression* right):
```

```
        BinaryExpression { left, right } {}
```

```
    double evaluate() override {
```

```
        return left->evaluate() - right->evaluate();
```

```
    }
```

```
};
```

```
f) class MathHomework {
```

```
private:
```

```
    std::vector<Expression*> expressions;
```

```
public:
```

```
    void addExpression (Expression* e) {
```

```
        expressions.push_back(e);
```

```
    }
```

```
    std::vector<double> getResults() const {
```

```
        std::vector<double> results;
```

```
        for (auto e: expressions)
```

```
            results.push_back(e->evaluate());
```

```
        return results;
```

```
    ~MathHomework() {
```

```
        for (auto e: expressions)
```

```
            delete e;
```

```
    }
```

```
};
```

```
g) int main() {
```

```
    MathHomework hw;
```

```
    hw.addExpression (new Adder { new Negate { new Constant { 5 } },
```

```
        new Subtractor { new Constant { 9 }, new
```

```
        Constant { 3 } } );
```

```
    hw.addExpression (new Subtractor { new Negate { new Adder {
```

```
        new Constant { 4 }, new Constant { 2 } } }, new Negate {
```

```
        new Constant { 10 } } );
```

```
    for (double result: hw.getResults())
```

```
        std::cout << result << " ";
```

```
    return 0;
```


Aliens

a) class Encoder {

public:

virtual std::string encode(const std::string& msg) = 0;

virtual ~Encoder() = default;

};

b) class AlienEncoder : public Encoder {

private:

std::string header;

public:

AlienEncoder(...): header{header} {}

std::string encode(const std::string& msg) override {

return ~~msg~~ header + msg;

}

};

c) class MorseEncoder : public Encoder {

public:

std::string encode(...) override {

std::string result;

for(char c: msg)

result += ".";

return result;

}

};

d) class Mixer : public Encoder {

private:

Encoder* e1;

Encoder* e2;

public:

Mixer(Encoder* e1, Encoder* e2): e1{e1}, e2{e2} {}

std::string encode(...) override {

std::string a = e1->encode(msg);

std::string b = e2->encode(msg);

std::string result;

int m = std::min(a.size(), b.size());

for (int i=0; i<m; i++) {

result += a[i];

result += b[i];

}

if (a.size() > b.size()) result += a.substr(m);

if (b.size() > a.size()) result += b.substr(m);

}

return result;

~Mixer() override {

delete e1;

delete e2;

}

};

e) class Communication {
private:

std::vector<std::string> messages;
Encoder* encoder;

public:

Communication(Encoder* encoder): encoder { encoder } {}
void addMessage ...

std::string communicate() {
std::sort(messages.begin(), messages.end());
std::string result;
for (const auto & msg: messages) {
result += encoder->encode(msg);
}
return result;
}

~ Communication

{ delete ... }

};

f) int main() {

Communication c1 { new MorseEncoder {} };

----- c2 { new Mixer { new AlienEncoder { "ET" },
new MorseEncoder {} } };

c1.addMessage("hi");
----- ("abc");

c2 - - - - ("hihihi");

- - - - ("abcabcabc");

std::cout << c1.communicate();

- - - - c2.communicate();

return 0;

}


```
class Section {
```

```
protected:
```

```
std::string title;  
std::string content;  
Checker* checker;
```

```
public:
```

```
Section(const std::string& title, const std::string& content, Checker* checker)  
: title {title}, content {content}, checker {checker} {}
```

```
virtual void addSection(Section* s) = 0;
```

```
std::string getTitle()...
```

```
std::string getContent()...
```

```
virtual void generate() {
```

```
if (checker -> check(this)) {
```

```
std::cout << title << " ";
```

```
std::cout << content;
```

```
} }
```

```
virtual ~Section() {
```

```
delete checker;
```

```
}
```

```
};
```

```
e) class Preface: public Section {
```

```
public:
```

```
Preface(const std::string& title, const std::string& content,  
Checker* checker): Section {title, content, checker} {}
```

```
void addSection(Section* s) override {
```

```
throw std::runtime_error("Preface cannot have sections");
```

```
}
```

```
};
```

```
f) class Chapter: public Section {
```

```
private:
```

```
std::vector<Section*> sections;
```

```
public:
```

```
Chapter(const std::string& title, const std::string& content,  
Checker* checker): Section {title, content, checker} {}
```

```
void addSection(Section* s) override {  
sections.push_back(s);
```

```
}
```

```
void generate() override {  
if (checker -> check(this)) {
```

```
std::cout << title...
```

```
<< content...
```

```
for (auto s: sections)
```

```
s -> generate();
```

```
} }
```

```
~Chapter() override {
```

```
for (auto s: sections)
```

```
delete s;
```

```
} }
```


3. Checker (2026 exam guide)

a) class Checker {

public:

virtual ^{bool} void check(Section* s) = 0;

virtual ~Checker() = default;

};

b) { class TitleChecker : public Checker {

public:

bool check(Section* s) override {

return s->getTitle().length() > 2;

}

};

class ContentChecker : public Checker {

public:

bool check(Section* s) override {

return !s->getContent().empty() && std::isupper(s->getContent()[0]);

}

};

c) class DoubleChecker : public Checker {

private:

Checker* c1;

Checker* c2;

public:

DoubleChecker(Checker* c1, Checker* c2) : c1{c1}, c2{c2} {}

bool check(Section* s) override {

return c1->check(s) && c2->check(s);

}

~DoubleChecker() override {

delete c1;

delete c2;

}

};

d) class Section

main() {

auto* preface = new Preface { "Title1", "Content1", new ContentChecker {} };

auto* chapter = new Chapter { "Chapter1", "Content2", new TitleChecker {} };

chapter → addSection(new Chapter { ~~new~~ "Subchapter1", "contents", new
new DoubleChecker { new TitleChecker {}, new ContentChecker {} } }); ✓ passes

chapter → addSection(new Chapter { ~~Subchapter 4~~ "invalid et",
new DoubleChecker { new TitleChecker {}, new ContentChecker {} } });

preface → generate();

chapter → generate();

delete preface;

delete chapter;

return 0;

}